

6 · Discovering governing equations from data (SINDy) ⇄

The previous notebooks all assumed a model *structure* and either fit its parameters or filled a gap with a neural network. This notebook goes further: given only a measured trajectory, can we recover the **symbolic equations of motion** themselves, with no equation supplied at all?

We use **SINDy** (Sparse Identification of Nonlinear Dynamics, Brunton, Proctor & Kutz 2016) via `DataDrivenDiffEq.jl`: build a large library of candidate nonlinear terms, then use sparse regression to keep only the handful that are actually needed. We apply it to the Lorenz system.

```
1 using DataDrivenDiffEq, DataDrivenSparse, ModelingToolkit, OrdinaryDiffEq, LinearAlgebra
   , Random, Plots
```

1. Generate trajectory data from the (secretly known) Lorenz system ⇄

In a real application this data would come from an experiment or a simulation whose equations we don't fully trust or don't have. Here we generate it from the classic Lorenz equations so we can check whether SINDy recovers them.

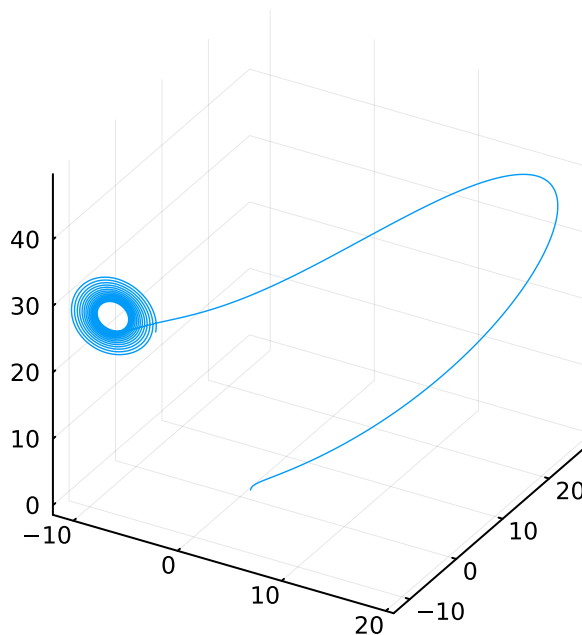
```
lorenz! (generic function with 1 method)
```

```
1 function lorenz!(du, u, p, t)
2      $\sigma$ ,  $\rho$ ,  $\beta$  = p
3     x, y, z = u
4     du[1] =  $\sigma$  * (y - x)
5     du[2] = x * ( $\rho$  - z) - y
6     du[3] = x * y -  $\beta$  * z
7     return nothing
8 end
```

```
► [0.0, 0.002, 0.004, 0.006, 0.008, 0.01, 0.012, 0.014, 0.016, 0.018, 0.02, 0.022, 0.024, 0.026,
```

```
1 begin
2   rng6 = Random.default_rng()
3   Random.seed!(rng6, 11)
4
5   lorenz_p6 = (10.0, 28.0, 8 / 3)
6   u0_6 = [1.0, 0.0, 0.0]
7   tspan_6 = (0.0, 10.0)
8   dt6 = 0.002
9
10  lorenz_prob6 = ODEProblem(lorenz!, u0_6, tspan_6, lorenz_p6)
11  lorenz_sol6 = solve(lorenz_prob6, Tsit5(); saveat=dt6)
12
13  X = Array(lorenz_sol6)                # 3 × N state trajectory
14  t6 = lorenz_sol6.t
15 end
```

Data given to SINDy (Lorenz trajectory)



```
1 plot(lorenz_sol6; idxs=(1, 2, 3), label=false, linewidth=0.7, title="Data given to  
SINDy (Lorenz trajectory)")
```

2. Wrap the data as a DataDrivenProblem ⇔

DataDrivenDiffEq.jl can estimate the derivatives $\dot{X}$ from the trajectory itself (numerical differentiation / collocation), so we don't even need to supply them directly — though supplying exact derivatives (as we do here, since we can compute them) gives a cleaner result for a first example.

```
Continuous DataDrivenProblem{Float64} ##DDProblem#296 in 3 dimensions and 5001 samples
```

```
1 begin
2     DX = similar(X)
3     for (i, ui) in enumerate(eachcol(X))
4         lorenz!(view(DX, :, i), ui, lorenz_p6, t6[i])
5     end
6     ddprob = ContinuousDataDrivenProblem(X, t6; DX=DX)
7 end
```

3. Build a candidate function library

SINDy needs a dictionary of candidate terms to choose from — here, all monomials up to degree 2 in (x, y, z) , which is more than enough to contain the true quadratic Lorenz right-hand side.

```
Model ##Basis#301 with 10 equations
```

```
States : x y z
```

```
Independent variable: t
```

```
Equations
```

```
 $\varphi_1 = 1$ 
```

```
 $\varphi_2 = x$ 
```

```
 $\varphi_3 = x^2$ 
```

```
 $\varphi_4 = y$ 
```

```
...
```

```
 $\varphi_{10} = z^2$ 
```

```
1 begin
2     @variables x y z
3     state_vars = [x, y, z]
4     basis = Basis(polynomial_basis(state_vars, 2), state_vars)
5 end
```

 Attempting to print a symbolic expression in a Pluto notebook. Please run ``import LaTeXify`` to enable pretty-printing of symbolic expressions. This warning will only display once.

4. Sparse regression

We search for the sparsest set of coefficients (over the candidate library) that still reproduces $\dot{X}$ accurately, using the STLSQ (sequential thresholded least squares) algorithm at a range of sparsity thresholds.

```
Model ##Basis#308 with 3 equations
States : x y z
Parameters : 7
Independent variable: t
Equations
Differential(t, 1)(x) = p1*x + p2*y
Differential(t, 1)(y) = p3*x + p4*y + p5*x*z
Differential(t, 1)(z) = p7*z + p6*x*y
```

```
1 begin
2     sparse_opt = STLSQ(exp10.(-4:0.25:1))
3     sindy_res = solve(ddprob, basis, sparse_opt)
4     sindy_sys = get_basis(sindy_res)
5 end
```

```
1 println(sindy_sys)
```



```
Model ##Basis#308 with 3 equations
States : x y z
Parameters : 7
Independent variable: t
Equations
Differential(t, 1)(x) = p1*x + p2*y
Differential(t, 1)(y) = p3*x + p4*y + p5*x*z
Differential(t, 1)(z) = p7*z + p6*x*y
```



Error message

MethodError: no method matching

`get_parameter_values(::DataDrivenDiffEq.DataDrivenSolution{Float64})`

The function `'get_parameter_values'` exists, but no method is defined for this combination of argument types.

Closest candidates are:

`get_parameter_values(::DataDrivenDiffEq.Basis)`

@ [DataDrivenDiffEq](#) ~/.julia/packages/DataDrivenDiffEq/YVjFe/src/basis/type.jl:642

Show stack trace...

```
1 get_parameter_values(sindy_res)
```

5. Validate: simulate the discovered model and compare ⇌

The discovered symbolic model can be converted straight back into an `ODEProblem` and simulated exactly like a hand-written one — closing the loop from data, to equations, to predictions.

Error message

```
MethodError: no method matching
get_parameter_values(::DataDrivenDiffEq.DataDrivenSolution{Float64})
The function `get_parameter_values` exists, but no method is defined for this
combination of argument types.

Closest candidates are:
  get_parameter_values(::DataDrivenDiffEq.Basis)
    @ DataDrivenDiffEq ~/.julia/packages/DataDrivenDiffEq/YVjFe/src/basis/type.jl:642
```

Show stack trace...

```
1 begin
2     discovered_prob = ODEProblem(sindy_sys, u0_6, tspan_6,
3     get_parameter_values(sindy_res))
4     discovered_sol = solve(discovered_prob, Tsit5(); saveat=dt6)
5     plot(lorenz_sol6; idxs=1, label="true x(t)", lw=2)
6     plot!(discovered_sol; idxs=1, label="SINDy-discovered x(t)", lw=2, ls=:dash)
7 end
```

Takeaways ⇌

- SINDy turns model discovery into a sparse regression problem: pick the fewest library terms that explain the observed derivatives.
- Because the result is a short symbolic expression rather than a black-box network, it is directly interpretable — you can read off the discovered coefficients and compare them to σ, ρ, β .
- This is the natural companion to the Universal Differential Equations notebook: train a neural network on the unknown part of a model, then run SINDy on the network's input/output pairs to turn it into a genuine equation.